

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO1 ASSESSMENT TOOL 2 – Agile Sprint Planning & User Story Workshop

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO1	Assessment:	Assessment Tool 2 – Agile Sprint Planning & User Story Workshop
Weightage:	20%	Total Marks:	25
CO1 Statement:	<i>Apply advanced methodologies and agile project management techniques to manage software projects effectively</i>		
Student Name:	N Goutham	Reg. No.:	192472024
Date:	24-07-2026	Duration:	60 minutes

Code of Conduct

I N Goutham(192472024) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: N Goutham

Annexure A – Agile Sprint Planning & User Story Workshop

Project Title:

Open-Source Contribution Tracker and Recommendation System for Beginner Developers

1. Project Vision and Stakeholder Identification

Project Vision

The vision of the Open-Source Contribution Tracker and Recommendation System is to create a centralized platform that helps beginner developers confidently start their open-source journey. Many beginners face difficulties in identifying suitable repositories, understanding contribution workflows, and tracking their progress on GitHub. The proposed system addresses these challenges by recommending beginner-friendly repositories, monitoring contribution activities, and generating a professional portfolio based on GitHub contributions.

The project aims to simplify the open-source contribution process through GitHub API integration, personalized recommendations, and contribution analytics. By reducing manual effort and providing continuous guidance, the system encourages students and beginner developers to improve their programming skills while building a strong professional profile for internships and career opportunities.

Project Objectives

The primary objectives of the project are:

- Help beginners discover suitable open-source repositories.
- Recommend repositories based on programming skills and interests.

- Integrate with the GitHub API for real-time repository and contribution data.
- Track pull requests, issues, and contribution history.
- Display learning progress through an interactive dashboard.
- Generate a professional portfolio highlighting open-source achievements.
- Encourage continuous participation in open-source communities.

Stakeholder	Role in the Project
Beginner Developers	Register, receive repository recommendations, contribute to projects, and track learning progress.
Project Maintainers	Review pull requests, manage repositories, and label beginner-friendly issues.
Mentors	Guide beginners by reviewing contributions and providing technical assistance.
Recruiters	Evaluate developers based on their contribution history and generated portfolio.
System Administrator	Manage users, monitor application performance, and maintain security.
GitHub Platform (API Provider)	Supplies repository, issue, pull request, and contribution data through GitHub APIs.

Stakeholders

Stakeholder Responsibilities

Beginner Developers

- Register and log in using GitHub.
- Create and manage user profiles.
- Contribute to recommended repositories.
- Monitor contribution progress.

Project Maintainers

- Manage repositories.
- Review pull requests.
- Assign beginner-friendly issues.
- Merge approved contributions.

Mentors

- Guide beginner developers.
- Suggest suitable repositories.
- Review coding practices.
- Encourage continuous learning.

Recruiters

- Review contribution portfolios.

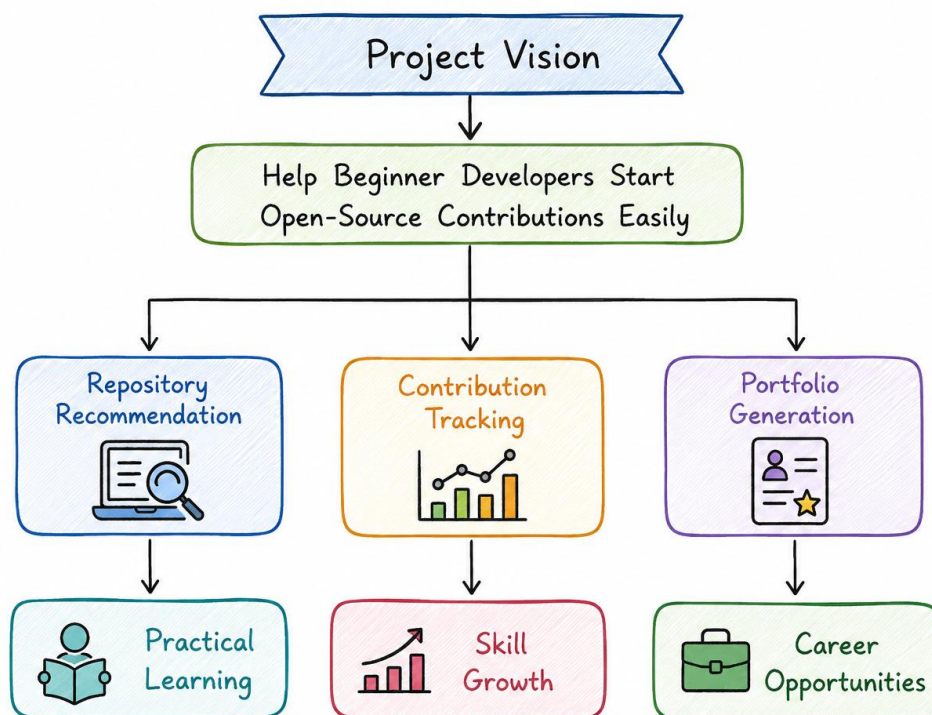
- Assess technical skills.
- Evaluate practical development experience.

System Administrator

- Manage user accounts.
- Monitor system security.
- Handle maintenance and updates.

GitHub Platform

- Provide repository information.
- Supply issue and pull request details.
- Authenticate users using GitHub OAuth.



2. Epics & User Stories

Introduction

In Agile software development, an Epic represents a large feature or business functionality that can be divided into multiple user stories. User Stories describe system requirements from the end user's perspective using the format "As a..., I want..., so that...". They help the development team understand user needs, prioritize features, and deliver the project incrementally through sprints.

2.1 Project Epics

Epic ID	Epic Name	Description
EP-1	User Authentication	Allow users to securely register and log in using GitHub OAuth authentication.
EP-2	User Profile Management	Enable users to create and manage their programming skills, interests, and experience level.
EP-3	GitHub Repository Recommendation	Recommend beginner-friendly repositories and issues based on the user's profile.
EP-4	GitHub API Integration	Retrieve repository, issue, pull request, and contribution data from GitHub.
EP-5	Contribution Tracking	Track pull requests, issues solved, merge status, and contribution history.
EP-6	Dashboard & Analytics	Display contribution statistics, learning progress, and technology usage.
EP-7	Portfolio Generation & Notifications	Generate professional portfolios and notify users about new beginner-friendly opportunities.
EP-8	Administration & System Management	Manage users, monitor system performance, and maintain application security.

2.2 User Stories

Story ID	Epic	User Story
US-01	EP-1	As a beginner developer, I want to log in using my GitHub account, so that I can securely access the application.
US-02	EP-2	As a beginner developer, I want to update my programming languages and technical skills, so that I receive personalized repository recommendations.
US-03	EP-2	As a beginner developer, I want to edit my profile whenever needed, so that my recommendations remain accurate.
US-04	EP-3	As a beginner developer, I want to receive beginner-friendly repository recommendations, so that I can start contributing with confidence.
US-05	EP-3	As a beginner developer, I want to filter repositories by programming language and technology, so that I can find projects matching my skills.
US-06	EP-3	As a beginner developer, I want to view issues labeled "Good First Issue", so that I can easily identify beginner-level tasks.

Story ID	Epic	User Story
US-07	EP-4	As a beginner developer, I want the system to retrieve repository information using the GitHub API, so that I always receive updated project details.
US-08	EP-5	As a beginner developer, I want to track my pull requests and issues, so that I can monitor my contribution progress.
US-09	EP-5	As a beginner developer, I want to view the status of my pull requests, so that I know whether they are open, closed, or merged.
US-10	EP-6	As a beginner developer, I want to view an interactive dashboard, so that I can monitor my learning progress and contribution statistics.
US-11	EP-6	As a beginner developer, I want to view technology usage and repository statistics, so that I can analyze my development growth.
US-12	EP-7	As a beginner developer, I want to generate a professional open-source portfolio, so that I can include it in my resume and LinkedIn profile.
US-13	EP-7	As a beginner developer, I want to receive notifications about newly available beginner-friendly issues, so that I never miss contribution opportunities.
US-14	EP-8	As an administrator, I want to manage user accounts, so that the application remains secure and organized.
US-15	EP-8	As an administrator, I want to monitor application performance and user activity, so that I can maintain system reliability.
US-16	EP-7	As a recruiter, I want to view a developer's contribution portfolio, so that I can evaluate their practical software development experience.

2.3 Mapping of Epics to User Stories

Epic	Associated User Stories
EP-1 – User Authentication	US-01
EP-2 – User Profile Management	US-02, US-03
EP-3 – GitHub Repository Recommendation	US-04, US-05, US-06
EP-4 – GitHub API Integration	US-07
EP-5 – Contribution Tracking	US-08, US-09
EP-6 – Dashboard & Analytics	US-10, US-11
EP-7 – Portfolio Generation & Notifications	US-12, US-13, US-16
EP-8 – Administration & System Management	US-14, US-15

3. Acceptance Criteria

Introduction

Acceptance Criteria define the conditions that must be satisfied before a user story is considered complete. They help developers, testers, and stakeholders understand the expected behavior of each feature. In Agile Scrum, the **Given–When–Then** format is commonly used to create clear, measurable, and testable acceptance criteria.

Acceptance Criteria for User Stories

US-01: GitHub Login

User Story

As a beginner developer,

I want to log in using my GitHub account,

So that I can securely access the application.

Acceptance Criteria (Given – When – Then)

Given the user has a valid GitHub account

When the user clicks the Login with GitHub button
--

Then the system should authenticate the user successfully and redirect them to the dashboard
--

US-02: Update User Profile

User Story

As a beginner developer,

I want to update my programming skills and interests,

So that I receive personalized recommendations.

Acceptance Criteria (Given – When – Then)

Given the user is logged in

When the user updates their skills and clicks Save

Then the profile should be updated successfully and used for future recommendations

US-03: Edit Profile

Acceptance Criteria (Given – When – Then)

Given the user is viewing their profile

When the user edits profile information and saves it
--

Acceptance Criteria (Given – When – Then)
--

Then the updated information should be stored successfully

US-04: Repository Recommendation

Acceptance Criteria (Given – When – Then)
--

Given the user has completed their profile

When the system retrieves repository information from GitHub

Then beginner-friendly repositories matching the user's skills should be displayed

US-05: Repository Filter

Acceptance Criteria (Given – When – Then)
--

Given repository recommendations are displayed

When the user selects a programming language filter
--

Then only repositories matching that language should be shown
--

US-06: View Good First Issues

Acceptance Criteria (Given – When – Then)
--

Given repositories are available

When the user selects Good First Issue
--

Then the system should display only beginner-friendly issues

US-07: GitHub API Integration

Acceptance Criteria (Given – When – Then)
--

Given GitHub API is available

When repository data is requested
--

Then the latest repositories, issues, and pull requests should be retrieved successfully

US-08: Contribution Tracking

Acceptance Criteria (Given – When – Then)
--

Given the user has contributed to a repository

When GitHub updates the contribution

Then the system should display the updated contribution history
--

US-09: Pull Request Status

Acceptance Criteria (Given – When – Then)
--

Given a pull request exists

When the system synchronizes with GitHub

Then the latest pull request status (Open, Closed, or Merged) should be displayed
--

US-10: Dashboard

Acceptance Criteria (Given – When – Then)
--

Acceptance Criteria (Given – When – Then)	
Given contribution data is available	
When the dashboard is opened	
Then contribution statistics and learning progress should be displayed	
US-11: Analytics	
Acceptance Criteria (Given – When – Then)	
Given contribution records are stored	
When analytics are generated	
Then repository statistics and technology usage should be displayed correctly	
US-12: Portfolio Generation	
Acceptance Criteria (Given – When – Then)	
Given the user has contribution history	
When the user clicks Generate Portfolio	
Then the system should create a professional portfolio containing contribution details	

4. Product Backlog Prioritization

Introduction

A Product Backlog is an ordered list of all the features, enhancements, and functionalities required to develop the software system. Each user story is prioritized according to its business value, customer needs, and dependencies. High-priority items are implemented first so that the core functionality of the application becomes available as early as possible.

4.1 Product Backlog

Backlog ID	User Story	Priority	Business Value	Dependency
US-01	Login using GitHub OAuth	High	Users cannot access the application without authentication.	None
US-02	Create and update user profile	High	Required for personalized recommendations.	US-01
US-03	Edit profile information	Medium	Allows users to keep their information updated.	US-02
US-04	Recommend beginner-friendly repositories	High	Core functionality of the application.	US-02, US-07
US-05	Filter repositories by programming language	Medium	Improves repository search experience.	US-04
US-06	Display "Good First Issue" tasks	High	Helps beginners identify suitable issues.	US-07
US-07	Retrieve repository data using GitHub API	High	Required to fetch repository information.	US-01

Backlog ID	User Story	Priority	Business Value	Dependency
US-08	Track pull requests and issues	High	Enables contribution tracking.	US-07
US-09	Display pull request status	Medium	Provides contribution updates.	US-08
US-10	Contribution Dashboard	High	Displays user progress and analytics.	US-08
US-11	Technology usage analytics	Medium	Helps users analyze learning progress.	US-10
US-12	Generate contribution portfolio	Medium	Useful for placements and internships.	US-10
US-13	Notification system	Low	Additional convenience feature.	US-04
US-14	Manage users	Medium	Required for administration.	US-01
US-15	Monitor application performance	Low	Helps maintain system reliability.	US-14
US-16	Recruiter portfolio view	Low	Additional feature for recruiters.	US-12

4.2 Prioritization Strategy

The backlog is prioritized using three important factors:

Business Value

Features that provide the greatest benefit to users, such as authentication, repository recommendation, and contribution tracking, are given the highest priority.

Customer Needs

The application is designed primarily for beginner developers. Therefore, features that directly improve their experience are developed before secondary functionalities.

Dependencies

Some modules depend on others. For example, repository recommendations require the user profile and GitHub API integration to be completed first. Therefore, dependent features are scheduled only after their prerequisites are finished.

5. Sprint Planning

Sprint Duration

- **Sprint Length:** 2 Weeks
- **Sprint Number:** Sprint 1
- **Team Size:** 4 Members (*or Individual Developer if applicable*)

Sprint Goal

Develop the core functionalities of the Open-Source Contribution Tracker by implementing secure GitHub authentication, user profile management, GitHub API

integration, and repository recommendation to enable beginners to discover suitable open-source projects.

5.1 Sprint Backlog

Task ID	Development Task	Mapped User Story	Owner	Status
T-01	Configure GitHub OAuth Login	US-01	Developer	Planned
T-02	Create User Profile Page	US-02	Developer	Planned
T-03	Profile Edit Functionality	US-03	Developer	Planned
T-04	Integrate GitHub REST API	US-07	Developer	Planned
T-05	Fetch Repository Data	US-07	Developer	Planned
T-06	Develop Recommendation Engine	US-04	Developer	Planned
T-07	Repository Filtering	US-05	Developer	Planned
T-08	Display Good First Issues	US-06	Developer	Planned
T-09	Unit Testing	All	Tester	Planned
T-10	Sprint Review & Documentation	All	Team	Planned

6. Sprint Goal & Expected Deliverables

6.1 Sprint Goal

The primary goal of Sprint 1 is to develop the core functionalities of the Open-Source Contribution Tracker and Recommendation System for Beginner Developers. By the end of the sprint, users should be able to securely log in using their GitHub account, create and manage their profile, retrieve repository information through the GitHub API, and receive beginner-friendly repository recommendations. These features establish the foundation for future sprints, which will include contribution tracking, analytics, portfolio generation, and notifications.

6.2 Expected Deliverables

At the completion of Sprint 1, the following deliverables are expected:

- GitHub OAuth authentication module.
- User profile creation and management feature.
- GitHub API integration for retrieving repository data.
- Beginner-friendly repository recommendation module.
- Repository filtering based on programming language.
- Display of "Good First Issue" repositories.
- Basic user dashboard with essential navigation.
- Unit testing for implemented features.
- Sprint documentation and project review.

6.3 Definition of Done (DoD)

A user story is considered **Done** only when all the following conditions are satisfied:

- The feature is fully implemented according to the requirements.

- The code has been reviewed and follows coding standards.
- Unit testing is completed successfully.
- The feature works correctly without critical bugs.
- The functionality is integrated with the existing system.
- Relevant documentation has been updated.
- The Product Owner accepts the completed feature.

6.4 Sprint Review

At the end of the sprint, the development team will demonstrate the completed features to the Product Owner and stakeholders. Feedback will be collected to verify that the implemented functionality meets user expectations. Any improvement suggestions or new requirements identified during the review will be added to the Product Backlog for future sprints.

6.5 Sprint Retrospective

After the Sprint Review, the Scrum team will conduct a Sprint Retrospective to evaluate the development process. The team will discuss what went well, identify challenges encountered during the sprint, and propose improvements for the next iteration. This continuous improvement process helps increase team efficiency and product quality throughout the project.

Conclusion

The Agile Sprint Planning process provides a structured approach to developing the Open-Source Contribution Tracker and Recommendation System for Beginner Developers. By defining a clear project vision, identifying stakeholders, creating epics and user stories, establishing acceptance criteria, prioritizing the product backlog, estimating story points, and planning sprint goals, the project is organized into manageable and achievable iterations.

Using the Scrum framework enables the development team to deliver working software incrementally while incorporating continuous feedback from stakeholders. This approach improves collaboration, adaptability, and software quality, ensuring that the final system effectively supports beginner developers in discovering open-source projects, tracking contributions, and building a professional development portfolio.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Project Vision & Stakeholder Identification	Project vision is clear, comprehensive, and aligned with the problem statement. All relevant stakeholders and their roles are correctly identified.	Vision is clear with minor omissions. Most stakeholders are correctly identified.	Vision is partially defined. Some stakeholders are missing or incorrectly identified.	Vision is unclear or incomplete. Stakeholders are poorly identified or missing.
Epics & User Stories	Epics are well-defined and user stories are complete, follow Agile format, and fully capture functional requirements.	Epics and user stories are mostly complete with minor formatting or requirement issues.	User stories are partially complete with limited Agile structure.	User stories are incomplete, unclear, or do not follow Agile practices.
Acceptance Criteria	Acceptance criteria are specific, measurable, testable, and aligned with all user stories.	Acceptance criteria are mostly clear and testable with minor gaps.	Acceptance criteria are partially defined and lack clarity or completeness.	Acceptance criteria are missing or not aligned with user stories.
Product Backlog Prioritization & Sprint Planning	Product backlog is logically prioritized, sprint backlog is realistic, and task breakdown is complete.	Backlog prioritization and sprint planning are mostly appropriate with minor issues.	Prioritization or sprint planning is partially complete with limited justification.	Backlog prioritization and sprint planning are poorly organized or incomplete.
Story Point Estimation, Sprint Goal & Presentation	Story point estimation is well justified, sprint goal is clear, deliverables are realistic, and presentation is professional and well organized.	Estimation and sprint goal are mostly appropriate with minor inconsistencies. Presentation is clear.	Estimation or sprint goal lacks justification. Presentation is adequate but incomplete.	Estimation is unrealistic or missing. Sprint goal, deliverables, and presentation are unclear or incomplete.

Criteria	Mark
Project Vision & Stakeholder Identification	/5
Epics & User Stories	/5
Acceptance Criteria	/5
Product Backlog Prioritization & Sprint Planning	/5
Story Point Estimation, Sprint Goal & Presentation	/5
TOTAL	/25